

Redwood Analytics Final Report

Waste Audit Collection and Storage System

By

Dean Callahan (Tech Lead)
Evan Blem (Domain Expert)
Zachary Griffiths (Project Manager)
Jaiden Roe (Quality Insurance)

Purpose:

Cal Poly Humboldt was recognized with a Bronze Zero Waste Certification by the Post-Landfill Action Network. As one of a few universities to earn this certification, it was important to understand how it had been achieved and how the university could continue to improve its waste practices. Campus waste audits were used to evaluate these waste practices on campus. The purpose of this project was to develop a more reliable and streamlined system for collecting, storing, and visualizing campus waste audit data. By reducing subjectivity in data entry, standardizing how data is stored, and improving accessibility to previous audits, our systems improved the immediate insights gained from each audit and the effectiveness of Cal Poly Humboldt's waste management practices.

Context:

Through meetings with Morgan King, the Office of Sustainability's Climate Action Analyst, the project team learned about the goals and challenges with Cal Poly Humboldt's waste audits. Mr. King's primary goal for the audits was to identify inefficiencies within the campus waste system in order to reduce total waste generation. The audits were also intended to raise awareness in the community about proper waste practices and to communicate findings to university administration and other groups. Mr. King's main challenge with the waste audits was the lack of a storage system for these audits which made comparing results over time difficult. In addition, a lack of consistency between audits limited the reliability of any findings. Because audits are conducted by volunteers with varying levels of experience and skills, the data collection process needed to be simple to use. Waste management practices also changed periodically at both the county and university levels, meaning that the collection and storage systems needed to be flexible enough to accommodate potential changes. Before the development of our system, waste audit data was collected manually by volunteers who recorded a limited number of data points. After each audit, the results were compiled into a new report, meaning that data was not stored for future access, except by informal methods.

Project Overview and Compliance with Scope & Plan:

To improve Cal Poly Humboldt's current waste audit system, we developed two prototype systems that the stakeholder can use to store and collect waste audit information. Throughout this report, both systems are compared and evaluated to determine which of these systems is the best long-term solution

The two systems we created were:

1. A Google Form connected to a Google Sheets spreadsheet
2. An Oracle SQL relational database connected to an HTML/PHP website form.

This aligned with our initial project plan of what systems we would develop and what technology/software we would use to make them. There were minimal deviations from the original plan. One deviation involved the methods used for data visualizations as we originally planned to use Python to create them but instead found built in spreadsheet features and SQL queries and tables to be sufficient. There were additional deviations that dealt with what data we tested our system on. We received some past waste audits from Morgan King but these didn't fit the standardized format we decided on. As a result, testing relied on the two audits we conducted personally and other data was mock or randomized values. Importantly, both of our systems collected the same types of observations in their data collection forms and contained the two real audits we conducted, that way they could be compared directly.

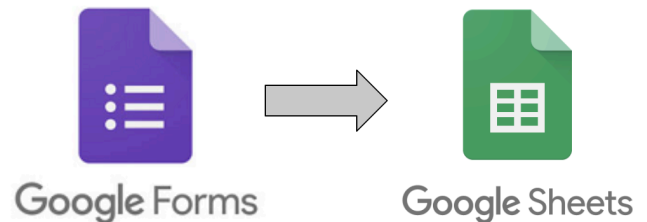
Prototype One: Google Sheets Spreadsheet

Link to the Google Form: <https://forms.gle/MPbx8TRMjiKBWCXq5>

Link to the Google Sheet: [📄 Waste Audit Form \(Responses\)](#)

System Information:

- We created our spreadsheet prototype using Google Forms to collect data and store it automatically in a Google Sheets spreadsheet (both linked above).
- Both the files for the Google Form and Google Sheets spreadsheet are located in a shared Google Drive folder so they could easily be shared with whoever would need to manage them in the future, such as Morgan King.



Data:

- Our sample data consisted of 10 different audits consisting of a mix of real data and mock data to show how the audits are stored in the spreadsheet structure. The first two rows are from real audits we performed on 9/11 and 9/25 and the rest were mock data we made. All of these audits were entered through our google form and then automatically imported into the spreadsheet. We did modify some values for the real audits manually in the spreadsheet (date and time as those columns are based off the time the form is submitted)

Timestamp	Audit Time	Building Location	Landfill Weight (# of lbs)	Mixed Recycling Weight (# of lbs)	Compost Weight (# of lbs)	Universal Waste Recycling Weight (# of lbs)	Hazardous Waste Weight (# of lbs)	Other Weight (# of lbs)
10/2/2025	17:24:50	SBS	10.91	5.26	22	0	0	2.5
10/2/2025	17:30:13	BSS	12.06598	0	9.243	2.38579	1.387	0.5
10/2/2025	17:32:51	LIB	5	10	8	7	2	8
10/2/2025	17:44:45	REDWOD	4.02387	13.98423	8.54353	11.52	14.0978	0
10/2/2025	17:46:42	JGC	17.61	21.11	50.39	14.23	0	1.11
10/2/2025	17:49:01	NR	11.42	3.5	33.83	0.1	0	0
10/2/2025	17:49:43	NR	11.42	3.5	33.83	0.1	0	0
10/2/2025	17:54:05	BALH	15.65464	6.6516	12.48321	5.636	8.097235	0.26
10/13/2025	15:04:43	ALDER	5	6	8	8	9	9

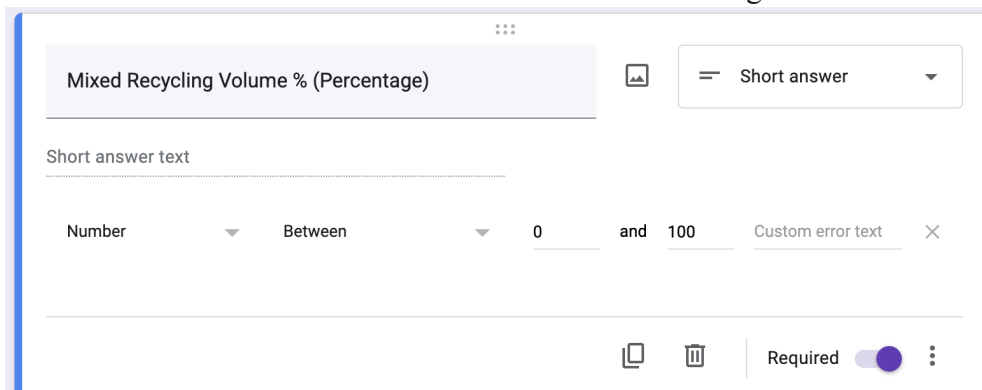
- Our goal with both systems but in particular the spreadsheet prototype was to reduce the amount of data the user had to enter. One advantage of the google form was it can automatically record certain metadata about the form user including email and a timestamp of when the form was

submitted. Additionally, using formulas in the spreadsheet we were able to calculate values like the percentage of weight for each category or total weight so the users did not have to calculate them.

-Refer to the data dictionary (Appendix D) for information about the data recorded by the spreadsheet prototype or click the spreadsheet quicklink at the top of this section to view the entire spreadsheet including all of its data.

Validations:

-Using the google forms and google sheets, we were able to enforce certain data validations for the waste audit. The first and most important validation was requiring certain questions be filled in google forms which we used to require the building location, the weights and percent volume for each waste category. Additionally, we decided other questions should be optional such as any comments or additional data like the number of landfill bags audited.

A screenshot of a Google Form editor showing a validation rule for a question titled "Mixed Recycling Volume % (Percentage)". The question type is set to "Short answer". Below the question, the validation rule is configured: "Number" (selected from a dropdown), "Between" (selected from a dropdown), with values "0" and "100" separated by "and". There is a field for "Custom error text" with a clear button (X). At the bottom right, there are icons for a copy, trash, and a toggle switch for "Required" which is currently turned on.

Other validations we implemented in this system related largely to the data types for questions and ensuring answers were logical. For example, in the image above depicts a percentage so we required the user enter a number between 0 and 100. Another example of a validation was ensuring the user chose an appropriate audit site by providing a dropdown list of all the potential campus sites which could be updated or modified easily in the future to account for new buildings or changes. This made sure all building locations were valid and logical audit sites.

System Limitations:

-There are not too many limitations for system use, as Google Forms and Google Sheets are very user friendly, and most students and staff will likely be familiar with how they work:

-One potential limitation with this system compared to our relational database is it is slower at processing large amounts of data but given the scope of the project this is not a major concern.

- One major limitation of the spreadsheet prototype in general is it is difficult to see relationships between data itself without using visualizations. For example, locating audits conducted on one building may become difficult when there are more audits whereas with the relational database you could query on that building location.

-Additionally, duplicate forms which submit the exact same data are possible in the spreadsheet format. However, they will likely have a different timestamp which can be used to uniquely identify them. These duplicate rows may skew certain visualizations or calculations. To reduce the possibility of duplicates we recommend it is enforced that only one person in each audit fills out the form.

-We also use formulas to compute certain values like the percent by weight for each landfill category. When someone edits the spreadsheet by adding or removing columns this can cause these computations to stop working. To reduce the risk of these breaking, ensuring only appropriate people have edit access is important for this system.

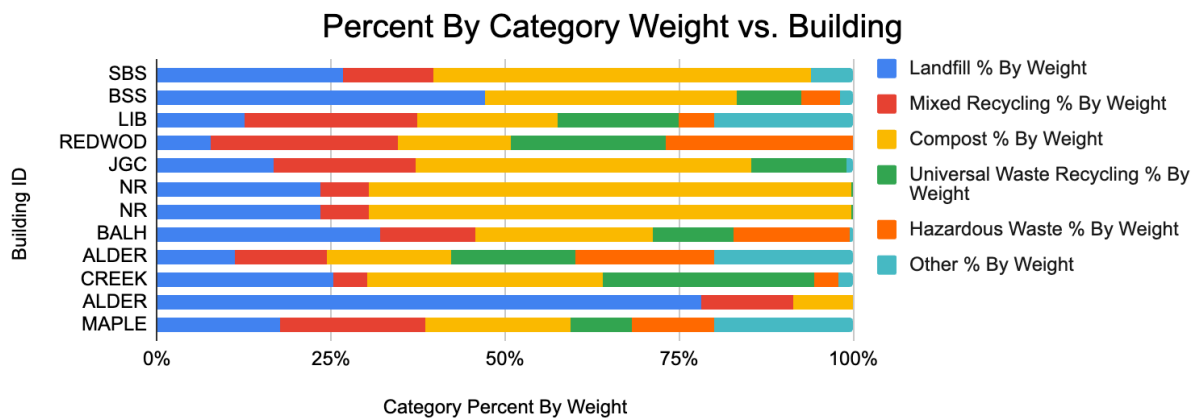
Reproducibility and Maintenance:

-For reproducibility we have labeled sheets to describe their function such as Waste Audit Responses, Visualizations, and Scalability and Performance testing. We have described some key validations in the spreadsheet within this section but refer to the data dictionary for an explanation of key rules or validations we enforced in both prototypes.

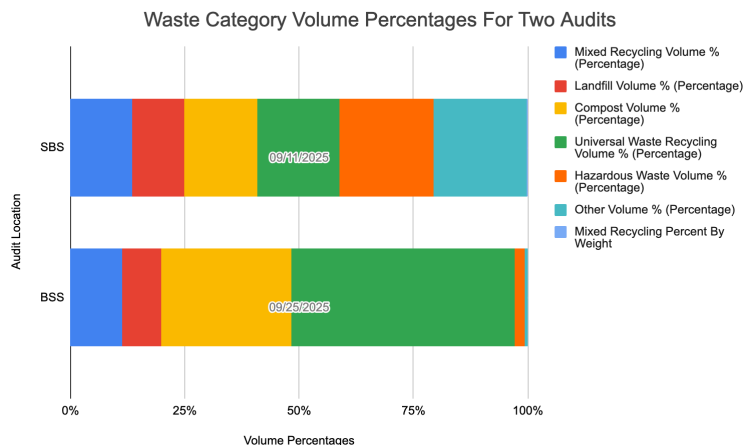
-Maintenance to this system is fairly straightforward as editors can directly change any cells if there are errors with the data such as if someone submitted a wrong value. An editor can also change the questions on the google form or add a new column in the spreadsheet to record additional information.

Sample Visualizations:

-This visual shows percent category weight for each audit in our spreadsheet. It can be used to visually compare audit data for each building.



-This visual directly compares two sample audits by comparing their volume percentages for the waste categories. Both of these visuals show more information when you hover over them in the spreadsheet like the exact percent for the categories.



Prototype Two: Relational Database using Oracle SQL

-Github for database files: <https://github.com/deancallahan25/Redwood-Analytics-Portfolio>

-Website for audit form: <https://nrs-projects.humboldt.edu/~dpc43/311project/form.php>

-Website for audit results: https://nrs-projects.humboldt.edu/~dpc43/311project/show_results.php

System Information:

-Our relational database was built using Oracle SQL. We created a design.sql file and a populate.sql which will create the database and populate it with some sample audit data once the files are run in SQLPLUS. Once the database exists on someone's system, you can visit the audit form website (ensure the link is correct based on the username and the file path) which we created with PHP and HTML and fill the information out which will push to the database. You can then fetch audit information either by running a query such as those located in the queries.sql file or by using our dynamic website which will display some selected audits and compare them.

Cal Poly Humboldt Waste Audit Form

By Redwood Analytics

[View Waste Audit Form Results](#)

What building did you conduct the audit on?

Landfill Weight (lbs): Landfill Volume (%):

Mixed Recycling Weight (lbs): Mixed Recycling Volume (%):

Compost Weight (lbs): Compost Volume (%):

Universal Waste Recycling Weight (lbs): Universal Waste Recycling Volume (%):

Hazardous Waste Weight (lbs): Hazardous Waste Volume (%):

Other Weight (lbs): Other Volume (%):

What are the most toxic items found?

Did you notice anything about the garbage collection systems in the building? (e.g., had dedicated paper towel bins in bathroom, had e-waste bin)

In general, do you have any recommendations about anything?

Date:

Time:

Auditor Information

Email: First Name: Last Name:

Optionally fill these in to improve the quality of the data collection:

Number of Bags Audited: Your calculated total weight:

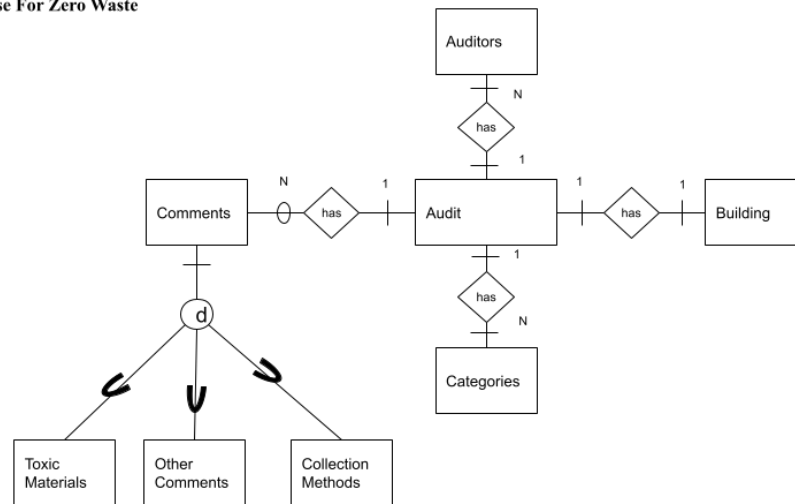
Design Information:

-Our database was based on our ERD model. The model includes all the tables that exist in our database and shows the appropriate relationship between them including primary and foreign keys. These tables and relationships were implemented in the design.sql file. Importantly, this file through the use of triggers, primary keys, foreign keys, and cascade constraints enforces referential integrity between tables.

Redwood Analytics - Dean Callahan, Evan Blem, Zachary Griffiths, Jaiden Roe

DATA 311 Project 1 - Database For Zero Waste

Date Last Modified: 10/12/24



Building	Waste_Audit	Auditors	Categories	Comments	Toxic_Materials	Other_Comments	Collection_Methods
<u>BUILDING_ID</u> (pk) building_name building_type (e.g. residential, faculty, food, student)	<u>AUDIT_ID</u> (pk) building_id (fk) number_bags total_weight weight_error volume_error date_conducted time_conducted	<u>AUDITOR_ID</u> (pk) email_address auditor_fname auditor_lname	<u>CATEGORY_NAME</u> (pk) category_weight percent_by_weight percent_by_volume audit_id (fk)	<u>COMMENT_ID</u> (pk) comment_type audit_id (fk)	<u>COMMENT_ID</u> (pk,fk) toxic_items (e.g. batteries, chemicals)	<u>COMMENT_ID</u> (pk, fk) other_comment (e.g. suggestions, issues)	<u>COMMENT_ID</u> (pk, fk) Collection_methods (e.g. type of garbage bins, e-waste bins, paper towels bin)

Data:

-We provided some sample data consisting of our actual audits we had conducted on 9/11 and 9/25 located in our populate.sql file. Additional mock data was entered through the website form and was the same mock data as our spreadsheet prototype. Importantly, only the populate.sql data will be saved until the system is transferred and set up with Campus IT or Morgan King.

-This prototype, similar to the spreadsheet prototype, also validates the data that is entered into our database. Using PHP and HTML form elements, the prototype is able to require certain question fields and make sure values entered are logical datatypes. Additionally, there are numerous triggers in the design.sql file which enforce referential integrity like one that automatically creates an auditor_id even if an insert statement leaves it blank. This HTML from our form shows some data validations like required values and numeric ranges.

```
</select>
<br />
<br />
<label for="landfill_weight">Landfill Weight (lbs):</label>
<input type="number" id="landfill_weight" name="landfill_weight" min="0.0" required="required"/>
<label for="landfill_volume">Landfill Volume (%):</label>
<input type="number" id="landfill_volume" name="landfill_volume" min="0.0" max="100" required="required"/>
<br />
```

System Limitations:

-One major limitation of this prototype is that it is difficult to make changes to the system. For example, in order to fix someone inputting an incorrect value into the form, someone with SQL experience would need to locate that response and write an update statement to fix it. This also applies to someone adding a new question like a new waste category as it would require someone with SQL knowledge and HTML/PHP knowledge to create a new question.

Ownership and Maintenance:

-Based on our discussions with Campus IT, Morgan King would need to host and manage the database on his account.

-Additionally, it is very important that the database is backed up as the Oracle service does get wiped occasionally by Campus IT meaning the audit data could be lost. To back up the data, the owner of the database could submit a ticket requesting they set aside your database not to be wiped. Additionally, they could use other SQL resources to export the data from the database at a set interval so they have multiple backup versions.

Sample Visualizations:

-Below is a sample query located in the queries.sql file which shows waste information for two audits in our database. This information could be copied to be used in a waste audit report. Refer to Appendix A for information about reproducing this query.

Creating query to compare audits 100001 and 100002

AUDIT_ID	BUILDING_ID	BUILDING_NAME	DATE_COND	CATEGORY_NAME	CATEGORY_WEIGHT	PERCENT_BY_WEIGHT	PERCENT_BY_VOLUME
100001	SBS	Student and Business Services	11-SEP-25	Landfill	10.91	25.9	30
100001	SBS	Student and Business Services	11-SEP-25	Mixed Recycling	5.26	13.1	18
100001	SBS	Student and Business Services	11-SEP-25	Compost	22	54.8	50
100001	SBS	Student and Business Services	11-SEP-25	Universal Waste Recycling	0	0	0
100001	SBS	Student and Business Services	11-SEP-25	Hazardous Waste	0	0	0
100001	SBS	Student and Business Services	11-SEP-25	Other	2.5	6.2	2
100002	MARKET	Marketplace Dining	25-SEP-25	Landfill	22.59	43.86	65
100002	MARKET	Marketplace Dining	25-SEP-25	Mixed Recycling	4.55	8.83	15
100002	MARKET	Marketplace Dining	25-SEP-25	Compost	24.36	47.37	20
100002	MARKET	Marketplace Dining	25-SEP-25	Universal Waste Recycling	0	0	0
100002	MARKET	Marketplace Dining	25-SEP-25	Hazardous Waste	0	0	0
100002	MARKET	Marketplace Dining	25-SEP-25	Other	0	0	0

12 rows selected.

-Additionally, there is a dynamic website application that can be used to compare two selected waste audits as a table. This also contains additional comparison information like the differences between categories' percentages and volumes. This can be accessed at the top of this section at the link to the website for waste audits results.

Cal Poly Humboldt Waste Audit Results

By Redwood Analytics

Select an Audit: Marketplace Dining (2025-09-25) Select Audit 2 (Optional): Boat Facility (2025-10-15) View Results

Audit ID	Building	Date Conducted	Category	Weight (lbs)	% by Weight	% by Volume
100002	Marketplace Dining	25-SEP-25	Compost	24.36	47.37	20
100002	Marketplace Dining	25-SEP-25	Hazardous Waste	0	0	0
100002	Marketplace Dining	25-SEP-25	Landfill	22.59	43.86	65
100002	Marketplace Dining	25-SEP-25	Mixed Recycling	4.55	8.83	15
100002	Marketplace Dining	25-SEP-25	Other	0	0	0
100002	Marketplace Dining	25-SEP-25	Universal Waste Recycling	0	0	0
100003	Boat Facility	15-OCT-25	Compost	10		50
100003	Boat Facility	15-OCT-25	Hazardous Waste	10		10
100003	Boat Facility	15-OCT-25	Landfill	10		10
100003	Boat Facility	15-OCT-25	Mixed Recycling	10		10
100003	Boat Facility	15-OCT-25	Other	10		10
100003	Boat Facility	15-OCT-25	Universal Waste Recycling	10		10
Comparison (Audit 100003 – Audit 100002)						
Compost Difference			Compost	-14.36 lbs	-47.37 %	30 %
Hazardous Waste Difference			Hazardous Waste	10 lbs	0 %	10 %
Landfill Difference			Landfill	-12.59 lbs	-43.86 %	-55 %
Mixed Recycling Difference			Mixed Recycling	5.45 lbs	-8.83 %	-5 %
Other Difference			Other	10 lbs	0 %	10 %
Universal Waste Recycling Difference			Universal Waste Recycling	10 lbs	0 %	10 %

Methods For Comparing the Systems

To evaluate which system of our two data collection and storage systems we recommended for Morgan King, we tested various criteria on both systems. These included the following:

Ease of Setup: How easy and quick it takes to set up each system is a moderately important factor to consider in which system we recommend. If one of our systems is put into use we probably only need to set up once for the Office of Sustainability or IT depending on who owns/maintains the systems. Since both of these systems are fully digital, this requirement applies exclusively to any software that must be installed for them. The only software needed for the database prototype is Oracle SQL database access and the ability to run setup files to configure the database. This ensures that Morgan King has access to the system and doesn't require any software installation. Morgan King would first have to download the required SQL files to his computer and execute the sql plus files, in order to set up the database. We estimate that it will take five to ten minutes to set up. Based on how long it took our team to set up the database with the tech lead's help, we calculated this number. Since the Google Spreadsheet database is already operational, there is no setup time required. All Morgan King would need to do is make sure he has edit access to the Google form and spreadsheet so he can make any necessary modifications. This was evaluated by our Project Manager and Tech Lead.

Ease of Use For Auditors: A key consideration when choosing a system is how easy it is for auditors to use. To do this we shared our input systems with three people of varying backgrounds including some familiar with google forms and some unfamiliar with it. For the forms of each system, we recorded how long it took the volunteers to enter the data, if it appeared in the form successfully, and if they had any issues with completing the form. For all three users they were able to fill out the forms within 10 minutes each once they had access to the links, they reported no issues with the process, and we saw their results in both storage systems. We also tested how both forms appear on mobile devices and across browsers to see if there were any major issues with inputting data. Both forms were able to be filled out across these different systems without issue. We believe that the google form is easier to use for auditors as more people are familiar with it and the UI is better due to it being easy to add visuals like the image for waste categories. This was evaluated by our Quality Assurance and Tech Lead.

Error Prevention & Data Quality: The accuracy and reliability of the waste data obtained are directly impacted by both error prevention and data quality, thus these are important considerations when choosing which solution to suggest. To test the effectiveness of each system against these criteria, we created a bad-data challenge set that we inputted into both forms and recorded if they were inserted into the respective data storage system. Additionally, for the SQL database system, we created a test_data.sql file which made sure our triggers and inserts worked correctly when certain values were left null. Screenshots of this file can be found in Appendix C or the file itself is located in the github link. For the spreadsheet system, it passed the majority of our bad-data challenge set blocking 14/17 tests. Our database system on the same bad-data challenge data set passed 13/17 tests. Our database system also passed all of our test_data.sql files successfully. Although it may seem like the spreadsheet is better, there are some alternative factors to consider. Built-in constraints and validation rules within the database structure make it easier to guarantee data integrity in the database prototype. For example, we had triggers that protect against certain missing values but also flag variables for weight and volume that tell us if there is an error within their respective waste categories. In the tests, where the database failed on blocking the bad data from being inputted it did raise a flag for two tests which would indicate in the database that this is bad data. Additionally, one major advantage of the spreadsheet system is after people submit their waste audit, several people can examine the data and easily modify incorrect data if they have edit access. This would be much more difficult in the SQL database as it would require someone with knowledge of SQL and need them to write queries to identify where the error is and update statements. Based on all these tests, we believe the spreadsheet system better meets this criteria than the SQL database. This was evaluated by our Quality Assurance, Domain Expert, and Tech Lead.

Ability to Handle Relationships: Relationships between data points are extremely important for gathering insights about waste practices and visualizing trends over time. Ensuring our data storage systems have relationships between data that can be modified as needed is a necessity for a proper database. We tested both our systems on this criterion. For the SQL relational database, we tested if we were able to implement our ERD, which had various entity relationships, such as each audit can have multiple auditors and multiple comments. We were successful in implementing our ERD relationships by using foreign keys and primary keys in our design.sql file, which created each entity class with referential integrity. For the spreadsheet, we were

unable to implement referential integrity and relationships outside of numeric calculations using formulas due to how a spreadsheet stores data. For this criterion, the relational database performed better than the spreadsheet system, as it was able to implement all our desired relationships with referential integrity, while the spreadsheet was not. This was evaluated by our Tech Lead and Quality Assurance.

Scalability & Performance: Although not a major concern given the small number of waste audits conducted yearly, how the two systems perform when lots of data is entered could influence which system we recommend. To test the scalability and performance of the relational database we timed how long it took to insert 5000 rows of mock data and

perform either a query on them to understand system performance. The relational database performed very well on this test as it was able to insert 5000 rows of randomly created mock data and then perform a simple query on them all within a second without any performance issues. To test the scalability of the spreadsheet we also inserted 5000 rows of mock data but based it off of two audits in our dataset instead of randomly generating it due to limitations of google sheets.

We were unable to use time as an appropriate test for the spreadsheet as we manually copied the rows until we reached 5000 making it irrelevant to record. We did notice significant performance issues with the spreadsheet when viewing or reloading the sheet with 5000 rows, likely due to it having to calculate some formulas and just the fact the spreadsheet system isn't meant to handle large datasets as well as a relational database. For this criteria, the relational database system performed much better than the spreadsheet system. This was evaluated by our Quality Assurance and Tech Lead.

```
SQL> start scalability_test.sql
Elapsed time: +000000000 00:00:00.138335000

PL/SQL procedure successfully completed.

Inserted the mock data

COUNT(*)
-----
5002

SQL> █
```

Reporting & Analysis: Our whole purpose with our data collection and storage system is to make it easier for Morgan King to analyze waste audits and report on them. We tested both our systems to see if they were able to display information comparing two audits that could be used in a waste audit report. For the spreadsheet prototype, google sheets has some built in charts and pivot tables features which we used to create visualizations comparing audits. In the database prototype we used queries to display information about the waste audits and also created a dynamic web application with a table comparing two selected audits. Both of these systems were able to meet our desired analysis and reporting levels as they compared and displayed information about at least two waste audits. Examples of these visualizations are included elsewhere in the report (in the sample visualization section for each system). For reports, given the fact that google sheets has built in charts you can create and creating visuals/analyses in the relational database requires SQL or HTML/PHP skills, the spreadsheet prototype is better for this criteria. This criteria was tested by the Tech Lead.

Integration with Campus IT: We meet with two workers from Campus IT to discuss the possible integration of both of our data collection systems with their current systems. Both teams for this project presented one of the types of data collection and storage systems and asked them about the logistics of integrating these systems. From our discussion, they strongly recommended we use the spreadsheet prototype for a few reasons. The first and most important is given limited staffing and the length of approval processes it is unlikely they would be able to integrate and maintain a database system. The second is there would be very minimal or no integration needed for the spreadsheet prototype as Morgan King would be able to maintain it himself. Additionally, they recommended that given the scope of our project with the limited amount of audits we would store the spreadsheet prototype would make more sense. Our Tech Lead, Quality Assurance, and Domain Expert carried out this discussion and presentation of the systems.

Cost (TCO): The total cost of ownership for each system is an important consideration for which system we recommend. To evaluate the costs for each system we recorded any software or hardware costs associated with the system and if it was necessary to have an employee responsible for maintaining and upgrading the system. For both systems, there was \$0 of software or hardware costs needed to own and operate them. For the spreadsheet prototype it is not necessary to have an employee to maintain or make upgrades to the system as this could entirely be handled by Morgan King with ease. For the relational database it is necessary to have knowledge of SQL and HTML/PHP to maintain or upgrade the system, especially given backups are necessary. This is outside the skill set of Morgan King so would require someone with this knowledge. This would likely require a school employee to spend a few hours every few months to maintain or upgrade the system, with a cost of \$16.50 (CA minimum wage) for those hours. For our estimate we will use two hours every quarter at a cost of \$16.50 for a total of cost of \$132 a year. This puts the cost of the relational database above the spreadsheet system meaning the spreadsheet system is best for this criteria. This was evaluated by the Project Manager and Tech Lead.

Maintenance after Setup: How much maintenance is required for each system is an important criteria for which system we recommend as we want it to be as simple as possible for Morgan King. To determine the maintenance for each system, we looked at how difficult it is to make changes to the system's design like adding or changing a question and if data needs to be backed up. For the spreadsheet prototype it is very simple to change or add a question and takes only a few minutes as you only need to modify the google form. Additionally, there is no need to back up data for this system. For the database prototype it takes much more effort and time to add or change a question. To modify or add a question, you at least need to change the html form and possibly add an attribute or table to a sql file. This takes around 15 minutes in our experience to complete when we already have knowledge of the domain and current system. Additionally, the database system needs to be backed up regularly. For this criteria the spreadsheet prototype performed much better as it was simpler and faster to make changes to the system and didn't require any data backups. This was evaluated by the Tech Lead.

Data and Quality Assurance Testing:

-Data Dictionary: Refer to Appendix D for the data dictionary

Scalability Test Dataset: Our scalability test used a 5000 row dataset in both our systems. In the relational database, our scalability test randomly generated these rows and its setup is located in the scalability_test.sql file (refer to Appendix B). In the spreadsheet system, our scalability test is located in the spreadsheet page named "Scalability & Performance Test" which uses two of our audits repeated multiple times to make up the 5000 rows.

Bad-data Challenge Set and Validation Outcomes:

Description	Data	Spreadsheet	Relational Database
Inputting invalid email address	notemail, 42, test.com	Passed, required and automatically filled in	Passed, required @ sign
Leaving audit date blank	Null value	Passed, always records timestamp for date	Passed, requires the field be filled out before submission,
Providing invalid date type/range	notadate, 12/12/2025 (date in the future)	Passed, can't enter an incorrect value as it records timestamp of when it was submitted	Passed 1/2 tests. Blocked first value as it you can't enter an invalid date type as form has date popup which only allows the correct date format. Allowed second value of date in the future
Leaving building location blank or invalid value	Null value, notabuildingid	Passed, required the field and only allowed you to choose preapproved building id	Passed, required the field and only allowed you to choose preapproved building i
User doesn't input a weight for a waste category	Null	Passed, required the user enter a value for all category weight questions	Passed, required the user enter a value for all category weight questions
User tries inputting an invalid range or datatype for waste category weight	-1, two, test	Passed 3/3, can't enter a number less than 0 and required it be of type number	Passed 3/3, can't enter a number less than 0 and required it be of type number
User doesn't input a percent for a waste category	Null	Passed, required the user enter a value for all category weight	Passed, required the user enter a value for all category weight

		questions	questions
User tries inputting an invalid range or datatype for waste category percent	-1, 200, two, test	Passed 4/4, value must be number between 0 and 100,	Passed 4/4, value must be number between 0 and 100,
User miscalculates total weight value	Incorrect weight value (category weights do not sum to this number)	Failed, allowed incorrect value with no indication it is wrong	Failed, allowed incorrect value to be entered in database but did raise weight_error flag
User volumes do not total to 100	Sum of volumes is not 100 or 1.0	Failed, allowed incorrect volumes with no indication it is wrong	Failed, allowed incorrect value to be entered in database but did raise volume_error flag
User enters duplicate form	duplicate data for all question	Failed, inserted duplicate form in sheet but timestamp differentiates them	Failed, inserted duplicate form in sheet but id's like audit_id or auditor_id differentiates them
TOTALS:		Blocked 14/17 tests	Blocked 13/17 of tests (Note although it failed more tests as it still allowed incorrect values to be pushed to the database it did raise error flags for two of the fails)

Final Analysis and Recommendation:

Our team recommends that Morgan King uses our **Google Form and Google Spreadsheet** for the data collection and storage of the waste audits. We based this recommendation off the following items:

Criterion	Spreadsheet	SQL Database
Ease of Setup	Zero software downloads required, just ensuring Morgan King has ownership/edit access for google form and spreadsheet. No real set up time.	Zero software downloads required, just ensuring the owner's account has access to nrs-projects. Requires importing design.sql and populate.sql and running via SQLPLUS. Setup takes ~5–10 minutes with Tech Lead assistance.
Ease of Use For Auditors	Users were able to fill out this form at the same speed as the other system's form with no issues and their results appeared in the spreadsheet. Contained better UI due to use of images of different categories. Performed well across devices and browser	Users were able to fill out this form at the same speed as the other system's form with no issues and their results appeared in the database. Performed well across devices and browser
Error Prevention and Data Quality	Passed 14/17 tests of our bad-data challenge set. Easy to fix errors with data sin system	Passed 13/17 of our bad-data challenge set but did raise error flags for some of them. Tested triggers on various insert statements in test_data.sql and passed all tests. Difficult to fix errors with data in system
Ability to Handle Relationships	Can handle some basic relationships between numeric values but no referential integrity	Can handle all our desired entity relationships based on our ERD diagram and has referential integrity.
Scalability & Performance	Tested by creating a 5000 row table sheet, performance was slow when viewing/reloading this sheet.	Able to insert and query on 5000 rows in <1 second with no performance issues.
Reporting & Analysis	Built-in chart and pivot table	Requires SQL queries or hard

	tools used to compare waste types across audits (see visualization section above for examples).	coding of PHP/HTML to create visualizations (see visualization section above or Appendix A for examples).
Integration with Campus IT	Uses google software, no integration required	Uses Oracle SQL database software, hosting/integration strongly preferred but not likely with Campus IT
Cost (TCO)	\$0 hardware/software cost. No additional staff maintenance required, can be handled entirely by Morgan King	\$0 hardware/software cost. Would require additional staff maintenance at an estimated cost of \$16.50/hr for 2 hrs every quarter.
Maintenance after Setup	Little maintenance required for spreadsheet. Changing or adding a question takes <5 minutes in Google Form. No backups required (Google auto-saves). Can be maintained and owned by Morgan King.	Some maintenance required. Takes ~15 minutes to modify or add a question. The database system needs to be backed up regularly. Can't be maintained by Morgan King but can be hosted/owned by him.

System Tradeoffs/Criterion Analysis and Recommendation based on Project Scope:

Overall, both systems have their advantages and disadvantages as a storage system for campus waste audits. From an operational perspective, the spreadsheet prototype is generally easier to set up, modify, and maintain based on our testing criteria. Additionally, it requires no cost to maintain, unlike the relational database. Both systems would provide the necessary reporting and analysis needed to create visuals in a report and identify potential changes to be made to campus waste practices. Both systems also performed well in error prevention and blocking bad data from being inputted in our system, with both of them blocking the majority of our tests, although the spreadsheet did slightly better. Additionally, both forms were satisfactory for the user's ease of use, although the spreadsheet is also slightly better due to it containing visuals for waste categories. Importantly, the spreadsheet does not need to be integrated with Campus IT, while the relational database does. Given our discussion with Campus IT, this integration is very unlikely, which makes recommending the relational database very difficult, as it would need frequent backups and maintenance, which Morgan King would be unable to do himself. Using the spreadsheet prototype does have some negative tradeoffs, as it is unable to maintain complicated relationships between values and has some performance issues related to scalability. In other settings, we would suggest the relational database, as it offers more complexity and better performance. However, due to the fact we only need to store a small number of waste audits yearly, we don't benefit from any advantages of the relational database in this situation. For these reasons we recommend that Morgan King use our spreadsheet prototype to collect and store Campus Waste Audits.

Cost & Maintenance Summary: The spreadsheet prototype has zero cost as all software is free and doesn't require an employee for system maintenance. This system would be completely owned and maintained by Morgan King/The Office of Sustainability as there is little maintenance required for the spreadsheet system except if changes to the questions or waste categories want to be made.

System Limitations: The spreadsheet prototype has some minor limitations:

- Data Size: The amount of data a google spreadsheet can hold is limited but this is not a serious concern given the limited amount of audits conducted per semester. However, spreadsheets having many audits with all the columns can cause certain features to be slowed down or impacted when lots of data is entered over time. This most likely will impact visualizations as when you have more audits, the visualization or table will become smaller and harder to read.
- Relationships: One major limitation of the spreadsheet prototype in general is it is difficult to see relationships between data itself without using visualizations. For example, locating audits conducted on one building may become difficult when there are more audits whereas with the relational database you could query on that building location.
- Additionally, duplicate forms which submit the exact same data are possible in the spreadsheet format. However, they will likely have a different timestamp which can be used to uniquely identify them. These duplicate rows may skew certain visualizations or calculations. To reduce the possibility of duplicates we recommend it is enforced that only one person in each audit fills out the form.
- We also use formulas to compute certain values like the percent by weight for each landfill category. When someone edits the spreadsheet by adding or removing columns this can cause these computations to stop working. To reduce the risk of these breaking, ensuring only appropriate people have edit access is important for this system.

Next Steps: The next step is to hand off the spreadsheet system to Morgan King if he chooses to use our system. We will share and transfer ownership of the google form and spreadsheet to Morgan King. Based on our volunteers being Cal Poly Humboldt students or staff, we can utilize SSO to restrict who has access to the google form. To do this we would simply restrict the google form access link to allow only Cal Poly Humboldt to view this link. Simple dashboards could be created either by creating more visualizations in google sheets with their chart and pivot tools or using other software. Further automation could be done to actually create the report by creating scripts and templates using the spreadsheet data and visualizations.

APPENDICES:

- A. Query that can be used to extract data and compare two audits. In sqlplus run the queries.sql file or this query directly to reproduce the output. This query could simply be modified with the id numbers to change which audits you compare.

```
--Select Information For Directly Comparing Audits (Compares Audit with ID 100001 and 100002)
--The two lines below will improve the format of the outputs
set linesize 170;
set pagesize 20;

prompt
prompt Creating query to compare audits 100001 and 100002
select w.audit_id, b.building_id, b.building_name, date_conducted, category_name, category_weight, percent_by_weight, percent_by_volume
from waste_audit w, categories c, building b
where b.building_id = w.building_id and w.audit_id=c.audit_id and w.audit_id in (100001, 100002);

set linesize 80;
set pagesize 14;
```

50,0-1

Bot

B. Database scalability_test.sql file

```
This file will insert 5000 rows into our database to test performance and scalability of this data storage system.
*/

--Create and time the amount it takes to insert 5000 mock rows in the waste_audit table
SET SERVEROUTPUT ON
DECLARE
    v_start TIMESTAMP;
    v_end TIMESTAMP;
BEGIN
    v_start:=SYSTIMESTAMP;
INSERT INTO Waste_Audit(audit_id, building_id, number_bags, total_weight, date_conducted)
SELECT waste_audit_seq.NEXTVAL,
    ((SELECT building_id FROM (SELECT building_id FROM building ORDER BY DBMS_RANDOM.VALUE) WHERE ROWNUM = 1)),
    round(DBMS_RANDOM.value(1,10)),
    round(DBMS_RANDOM.value(5,200),2),
    trunc(SYSDATE-DBMS_RANDOM.VALUE(1,365))
FROM DUAL
CONNECT BY LEVEL<=5000;

v_end := SYSTIMESTAMP;
DBMS_OUTPUT.PUT_LINE('Elapsed time: ' || TO_CHAR(v_end-v_start));
END;
/

prompt Inserted the mock data

select count(*) from waste_audit;

~
~
```

C. test_data.sql file. This file tested inserts, triggers, and referential integrity, it was successful as all values were entered into the database

```
1
2 --Testing all the triggers from the design sql statements
3
4
5 -- Building
6
7 INSERT INTO Building (BUILDING_ID, building_name, building_type)
8 VALUES ('B001', 'Science Hall', 'Academic');
9
10 INSERT INTO Building (BUILDING_ID, building_name, building_type)
11 VALUES ('B002', 'Main Library', 'Academic');
12
13 -- waste audits
14
15 INSERT INTO Waste_Audit (building_id, number_bags, total_weight, weight_error, volume_error, date_conducted)
16 VALUES ('B001', 10, 25.00, NULL, NULL, TO_DATE('2025-10-10', 'YYYY-MM-DD'));
17
18 INSERT INTO Waste_Audit (building_id, number_bags, total_weight, weight_error, volume_error, date_conducted)
19 VALUES ('B002', 5, 12.50, NULL, NULL, TO_DATE('2025-10-11', 'YYYY-MM-DD'));
20
21 -- auditors
22
23 INSERT INTO Auditors (email_address, auditor_fname, auditor_lname)
24 VALUES ('dcallahan@redwood.edu', 'Dean', 'Callahan');
25
26 INSERT INTO Auditors (email_address, auditor_fname, auditor_lname)
27 VALUES ('zgriffiths@redwood.edu', 'Zachary', 'Griffiths');
28
29 -- categories
30
31 INSERT INTO Categories (category_name, audit_id, category_weight, percent_by_weight, percent_by_volume)
32 VALUES ('Recyclables', 100003, 10.00, 40.00, 40.00);
33
34 INSERT INTO Categories (category_name, audit_id, category_weight, percent_by_weight, percent_by_volume)
35 VALUES ('Compost', 100003, 10.00, 40.00, 40.00);
36
37 INSERT INTO Categories (category_name, audit_id, category_weight, percent_by_weight, percent_by_volume)
38 VALUES ('Landfill', 100003, 5.00, 20.00, 20.00);
39
```

D. Data Dictionary:

- **Building:** string/text data representing the CPH campus map abbreviations for building names
 - Set this up using a dropdown menu so that input mistakes are minimized
 - This includes old labels such as for buildings that were torn down as there may have been audits conducted as these locations in the past
- **Landfill Weight (lbs):** numerical float data representing the weight in pounds of landfill found in the audit
- **Landfill Volume (%):** numerical float data representing the percentage of landfill in the total audit
 - Valid range: 0 – 100
- **Mixed Recycling Weight (lbs):** numerical float data representing weight in pounds of mixed recycling found in the audit
- **Mixed Recycling Volume (%):** numerical float data representing the percentage of the total audit volume that consists of mixed recycling
 - Valid range: 0 – 100
- **Compost Weight (lbs):** numerical float data representing the weight in pounds of compost found in the audit
- **Compost Volume (%):** numerical float data representing the percentage of the total audit volume that consists of compost
 - Valid range: 0 – 100

- **Universal Waste Recycling Weight (lbs):** numerical float data representing the weight in pounds of universal waste recycling found in the audit
- **Universal Waste Recycling Volume (%):** numerical float data representing the percentage of the total audit volume that consists of universal waste recycling
 - Valid range: 0 – 100
- **Hazardous Waste Weight (lbs):** numerical float data representing the weight in pounds of hazardous waste found in the audit
- **Hazardous Waste Volume (%):** numerical float data representing the percentage of the total audit volume that consists of hazardous waste
 - Valid range: 0 – 100
- **Other Weight (lbs):** numerical float data representing the weight in pounds of any other types of waste found in the audit
- **Other Volume (%):** numerical float data representing the percentage of the total audit volume that consists of any other types of waste found in the audit
 - Valid range: 0 – 100
- **Date:** date of when the audit was recorded
 - The user selects a date from a calendar popup or it's recorded by the form itself to ensure it is always in a consistent format.
- **Time:** Timestamp data of when the audit was recorded
 - The user selects the time from a time popup or it's recorded by the form itself to ensure it is always in a consistent format
- **Email:** email address of the person who submitted the form
 - This is automatically recorded by the google form so it is in proper email format
 - The relational database uses the email input type to ensure it is in the proper email format
- **First Name:** String of the person's first name
- **Last Name:** String of the person's first name
- **Number of Bags Audited:** The number of bags that were went through in the waste audit
 - Numerical integer value
 - Optional
- **Calculate Total Weight:** This is the total weight of all the waste categories calculated by the user. This number is used to compare to the computed total value to flag a potential error with the weights entered
 - Numerical float value
 - Optional (we don't require this as we can calculate it with the other category weights)

Note: There may be some personal data collected from the waste audits forms, although these fields are mostly optional. For instance, we collect auditor information, including their first name, last name, and their email address. The email address is required for the google form but the auditor's name is always optional in both forms.